

LAB WORKBOOK

Assignment 6: Windows, Tabs, and Iframes

Tier 2: Advanced User Interactions

1. Module Name and Experiment Title

Module: Tier 2 - Advanced User Interactions

Experiment: Windows, Tabs, and Iframes

2. Problem Statement

Automate a web page that contains an embedded iframe and a control that opens a new browser tab or window. Interact with an element inside the iframe by switching to it with `driver.switch_to.frame()`. Then open the new tab, use `driver.window_handles` to switch to the new browsing context, retrieve its title, close the new tab, and switch back to the main layout.

3. Objective

To learn how Selenium manages multiple browsing contexts and embedded frames, and to practice switching between the main document, an iframe, and a newly opened browser tab.

4. Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Browser	Google Chrome
IDE/Editor	VS Code / PyCharm
Practice Pages	The Internet - Frames and Windows examples
Concepts	iframe, <code>switch_to.frame()</code> , <code>switch_to.default_content()</code> , <code>window_handles</code> , <code>switch_to.window()</code> , <code>title</code> , <code>close()</code>

5. Source Code / Implementation

The following implementation uses The Internet practice site. The frame section demonstrates switching into an iframe and interacting with its content. The windows section opens a new browser tab, retrieves its title, closes it, and returns to the original tab.

```
import time

from selenium import webdriver
```

```
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()

try:
    driver.get("https://rahulshettyacademy.com/AutomationPractice/")

    main_window = driver.current_window_handle

    # 1. Switch to Iframe and interact
    driver.switch_to.frame("courses-iframe")

    iframe_text = driver.find_element(
        By.XPATH,
        "//*[@class='headerupper']/li[contains(text(), 'contact')]"
    ).text

    print(f"Extracted from Iframe: {iframe_text}")

    # Switch back to default content
    driver.switch_to.default_content()

    # 2. Open new tab and switch
    driver.find_element(By.ID, "opentab").click()

    time.sleep(2) # Allow tab to load
```

```
all_windows = driver.window_handles

for window in all_windows:
    if window != main_window:
        driver.switch_to.window(window)
        break

print(f"New Tab Title: {driver.title}")

# Close the child tab
driver.close()

# Switch back to main window
driver.switch_to.window(main_window)

print(f"Returned to Main Window Title: {driver.title}")

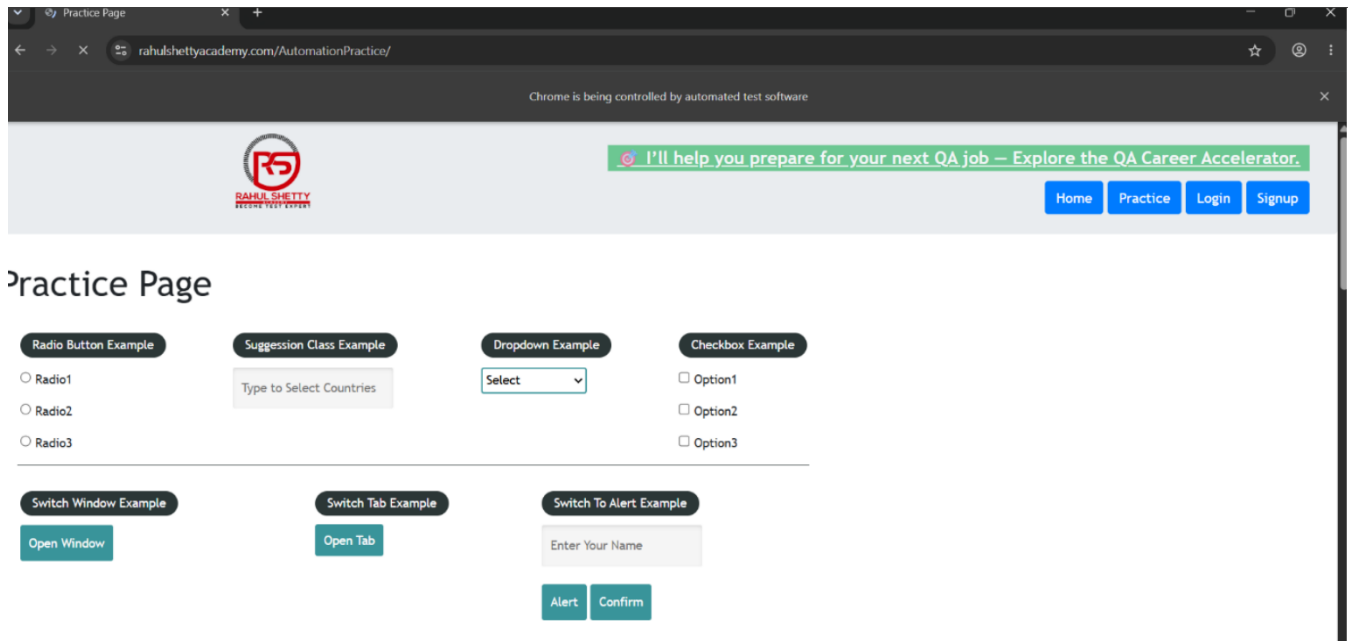
finally:
    driver.quit()
```

6. Implementation Steps

1. Open the iframe practice page in Chrome.
2. Locate the iframe element and use `driver.switch_to.frame()` to enter its context.
3. Locate the editable element inside the iframe and type text into it.
4. Use `driver.switch_to.default_content()` to return to the main document.
5. Open the windows practice page and store the current tab handle using `driver.current_window_handle`.
6. Click the control that opens a new tab/window.

7. Use `driver.window_handles` to obtain the available browsing contexts.
8. Switch to the new tab using `driver.switch_to.window()`.
9. Read and print the new tab's title.
10. Close the new tab and switch back to the original tab.

7. Output



8. Result

The iframe was accessed successfully using `switch_to.frame()`, the required element was manipulated, and the browser context was returned to the main document. A new tab was also opened, its title was retrieved, it was closed, and control was switched back to the original tab.

9. Observation

Selenium treats an iframe as a separate document context, so elements inside it cannot be accessed until the driver switches into the frame. Similarly, multiple tabs or windows require window handles to identify and switch between browsing contexts.

10. Conclusion

This experiment demonstrated Selenium handling of iframes and multiple browser windows/tabs. The use of `switch_to.frame()`, `default_content()`, `window_handles`, `switch_to.window()`, and `close()` enables reliable navigation between different web contexts.